OOP Concepts in Secrets of Ravenwood

A simple guide to where each concept appears in the current C++ source

Source file: **src/main.cpp**

1. Abstraction

The `Challenge` class defines what every challenge must provide, without specifying how it creates its questions. Its pure virtual methods make it an abstract base class. (Lines 28-34)

```
class Challenge {
 public:
  virtual ~Challenge() = default;
  virtual const std::vector<Question>& pool() const = 0;
  virtual int count() const = 0;
  virtual std::string name() const = 0;
};
```

2. Inheritance

`RiddleChallenge` and `MealChallenge` inherit from `Challenge` and provide their own implementations. (Lines 35 and 44)

```
class RiddleChallenge final : public Challenge { ... };
class MealChallenge final : public Challenge { ... };
```

3. Polymorphism

The game stores different challenge types as `Challenge*` pointers, then calls `pool()` through that shared type. The actual challenge object supplies the appropriate implementation. (Lines 111-115)

```
static const Challenge* stages[] = {
  &hall, &bedroom, &kitchen, &library
};
auto pool = stages[stage]->pool();
```

In practice: the same question-selection logic can handle both riddles and the Kitchen food challenge.

More OOP ideas in the code

Objects, composition, and an important accuracy note

4. Classes and objects

`Challenge`, `RiddleChallenge`, and `MealChallenge` are classes. `hall`, `bedroom`, `library`, and `kitchen` are objects created from those classes. (Lines 28-60 and 62-92)

```
const RiddleChallenge hall({ /* question data */ }, "Hall Riddles");
const MealChallenge kitchen;
```

5. Composition (a "has-a" relationship)

A `Game` contains question collections in its `chosen` member. This is a simple example of composition/containment. (Lines 94-100)

```
struct Game {
  int score = 0, lives = 5;
  std::vector<Question> chosen[4];
};
```

What about encapsulation?

The challenge classes keep their question data in `private` members. But `Game` is a `struct` with public fields, so encapsulation is only partial in this code. (See lines 35-43 and 94-101.)

```
private:
  std::vector<Question> questions_;
  std::string name_;
```

Presentation summary: The clearest examples are abstraction, inheritance, and polymorphism in the Challenge classes. The code also uses classes and objects, and composition in Game. Encapsulation appears in the challenge classes, while Game exposes its data publicly.